

SmartFight Phase 1 — Boxing Migration & Foundation

Date: 2026-04-24

Status: Approved for implementation

Scope: Foundation layer for boxing-oriented fan experience

Part 1 — Plain Language Summary

What this project is becoming

SmartFight started as an MMA-oriented platform inspired by UFC. This phase pivots the product toward **professional boxing**, while preserving the UFC-style user experience that fans are already familiar with. The application will behave like a boxing fan's companion site: a place to see upcoming fights, browse past results, check the rankings in each weight class, predict outcomes, and read editorial content.

Nothing that currently works for fans goes away. The blog, predictions, leaderboard, and notification systems remain untouched. What changes is the shape of the underlying data — how fights, fighters, events, and divisions are modeled — so that the site speaks the language of boxing rather than MMA.

What's changing for visitors

A person visiting the site will, once this phase and the phases that follow are complete, be able to create a fan account through a new sign-up page, browse a calendar of upcoming boxing events with full fight cards and betting odds, look at past events with results and knockdown counts, view official rankings for every weight class showing the reigning champion and the top ten contenders, see each fighter's professional record, nationality, and photo, and follow all of this with the same visual polish as ufc.com.

Nothing about account-level security is being weakened. Sign-up creates fans only; there is no way for a visitor to grant themselves administrator privileges through the public interface.

What's changing for administrators

Administrators will continue to have full control over the site's content, but the tools they work with are being reshaped for boxing. They'll manage fighters (with photos, nationality, and weight class), events (with real venues, cities, countries, and optional posters), individual fights within an event (with scheduled rounds, title-fight designation, and pre-fight odds), and post-fight results (including per-round punch statistics and knockdown counts).

Much of what an administrator previously had to do by hand becomes automatic. Event status — whether an event is scheduled, live, completed, or cancelled — will be derived from the data rather than manually set. Administrators will only need to intervene to cancel an event; everything else transitions on its own as dates pass and results are entered.

Entering post-fight statistics, historically the most tedious administrative task, becomes flexible. For a routine fight, an administrator can enter only the totals — four numbers — and be done. For a marquee fight where detailed per-round data matters, a full grid is available. Either format can be imported or exported as a CSV file, so administrators working in Excel can handle the numbers in whichever tool they prefer.

What's changing under the hood

The database is gaining new concepts that didn't exist before: weight classes as first-class entities (with seventeen boxing divisions seeded from the start), rankings as their own data structure, per-fight statistics with optional per-round breakdowns, and country codes for flag display. Legacy fields that were left over from the MMA matchmaking origins of the project — like free-text weight class strings, duplicate ID columns, and event-level championship flags — are being cleaned up or deprecated.

The fight result methods are being changed from MMA terminology (KO/TKO, Submission, Decision) to boxing terminology (KO, TKO, UD, SD, MD, PTS, DQ, TD, NC, Draw). The scheduled number of rounds becomes configurable per fight, with the standard boxing values of 4, 6, 8, 10, or 12 rounds available.

Odds are stored in decimal format — the standard in Tunisia, Europe, and most of the world outside the United States — and displayed as decimal throughout the interface.

What's not changing in this phase

No user-facing features are being built in Phase 1. This phase establishes the data foundation: new database tables, modified entities, enumerated constants, a sign-up page, a console command for creating the first administrator, and the background logic that keeps event statuses current. The administrative screens for managing fighters, events, and matches come in Phase 2. The public pages for browsing events and viewing rankings come in Phases 3 through 5.

Phase 1 is deliberately scoped so that when it completes, the project still looks the same to an end user — but everything built on top of it in subsequent phases will be built on solid, boxing-appropriate foundations.

Why this structure

Two principles shaped every decision in this plan.

First, **security is not negotiable**. The initial proposal to let visitors choose "admin or fan" at sign-up was rejected immediately because it would have made the entire access control system meaningless. The first administrator is created through a command-line tool that only someone with server access can run; subsequent administrators are promoted by existing ones. This is the standard pattern for a reason.

Second, **automation reduces administrative burden wherever it's safe to do so**. Event status transitions happen automatically based on dates and data. Fight stats entry offers both a minimum-effort path (totals only) and a detailed path (per-round), with CSV import/export as a bridge between the web interface and spreadsheet workflows. Rankings remain manual because subjective judgment belongs to humans — the UFC doesn't compute its rankings from a formula, and neither should this project.

Part 2 — Technical Specification

Section 1: New Entities

1.1 WeightClass

Represents a single boxing weight division.

Field	Type	Notes
id	integer, PK	
name	string(100)	e.g. "Heavyweight"
slug	string(100), unique	e.g. "heavyweight"
weightLimitLbs	integer, nullable	NULL for Heavyweight (no limit)
weightLimitKg	decimal(5,2), nullable	NULL for Heavyweight
displayOrder	integer	For presentation ordering
discipline	FK → Discipline	NOT NULL
champion	FK → Fighter	nullable (null = vacant title)

Seeded at migration time with seventeen boxing divisions linked to the Boxing discipline. Champion slot begins NULL for all.

1.2 Ranking

Represents one fighter holding one position in one weight class's top ten.

Field	Type	Notes
id	integer, PK	
weightClass	FK → WeightClass	NOT NULL
fighter	FK → Fighter	NOT NULL
position	integer (1-10)	
updatedAt	datetime	

Unique constraint on (weightClass, position) and (weightClass, fighter) .

1.3 FightStat

Per-round (or per-fight-total) statistics for one fighter in one fight.

Field	Type	Notes
id	integer, PK	
fightResult	FK → FightResult	NOT NULL
fighter	FK → Fighter	NOT NULL
roundNumber	integer, nullable	NULL = fight totals; 1-12 = per-round
punchesThrown	integer, default 0	
punchesLanded	integer, default 0	
knockdowns	integer, default 0	

Unique constraint on (fightResult, fighter, roundNumber) — enforced at application level due to NULL handling.

Data rules: every fight must have exactly two totals rows (one per fighter, roundNumber = null); per-round rows are optional, 0 or N per fighter where $N \leq \text{scheduledRounds}$.

1.4 SystemMeta

Single-row table for system-level timestamps and flags.

Field	Type	Notes
id	integer, PK	
key	string(100), unique	e.g. last_status_refresh
value	string(255)	
updatedAt	datetime	

Section 2: Modified Entities

2.1 Fighter

Added fields

Field	Type	Notes
firstName	string(100), nullable	For fighters not linked to a User
lastName	string(100), nullable	For fighters not linked to a User
countryCode	string(2), nullable	ISO 3166-1 alpha-2, for flag display
weightClass	FK → WeightClass, nullable	Replaces legacy fields
discipline	FK → Discipline, NOT NULL	Defaults to Boxing

Modified fields

- user FK changed from NOT NULL to nullable

Dropped fields

- weightClassId (legacy integer column, no FK)
- weightClass (legacy free-text string column)

Untouched (legacy, keep as dead weight)

- eloRating, performanceScore, strengthOfSchedule, winStreak, championsEventWinStreak, titleDefenses, submissionWins, decisionWins
- nationality string (distinct from new countryCode)
- wins, losses, draws, koWins (already served our needs)

Display name behavior update: getDisplayName() returns composed User-based name if User is set; otherwise returns firstName lastName with optional nickname.

Photo handling: continues using existing VichUploaderBundle fighter_photo mapping.

2.2 Event

Added fields

Field	Type	Notes
startsAt	datetime	Replaces date-only startDate
endsAt	datetime	Replaces date-only endDate
venueName	string(150), nullable	
city	string(100), nullable	
country	string(2), nullable	ISO code
posterUrl	string(255), nullable	VichUploader managed

Modified fields

- `status` converted to enum-constrained string (values from `EventStatus` enum)

Dropped fields

- `startDate` (date-only, data migrated to `startsAt` with 00:00:00 time)
- `endDate` (date-only, migrated similarly)
- `venueId` (dead integer column)
- `organizerId` (dead integer column)
- `isChampionsEvent` (wrong granularity — moved to match level)
- `location` (replaced by structured fields)

Photo handling: new VichUploader `event_poster` mapping.

2.3 MatchProposal

Added fields

Field	Type	Notes
<code>scheduledRounds</code>	integer	Allowed: 4, 6, 8, 10, 12
<code>isTitleFight</code>	boolean, default false	
<code>weightClass</code>	FK → WeightClass	NOT NULL
<code>oddsFighter1</code>	decimal(5,2), nullable	Decimal format
<code>oddsFighter2</code>	decimal(5,2), nullable	Decimal format
<code>cardPosition</code>	integer	1 = main event
<code>cardType</code>	string enum	MAIN_CARD or PRELIMS

Modified fields

- `status` values normalized to `MatchStatus` enum set; existing `'PENDING'` rows migrated to `'SCHEDULED'`

Dropped fields

- `compatibility` (assumed dead; verified during implementation, kept if in use)

2.4 FightResult

Added fields

Field	Type	Notes
<code>knockdownsFighterRed</code>	integer, default 0	
<code>knockdownsFighterBlue</code>	integer, default 0	

Modified fields

- `method` enum-constrained to `FightMethod` values

Untouched

- `status` (remains 'COMPLETED' default)
- Direct `event` FK (denormalization preserved)
- Nullable `match` FK (flexibility preserved)
- `fightNumber` (serves as historical card position)

Section 3: Enumerations

All implemented as PHP classes with public string constants. No native PHP 8.1 enums (consistency with existing codebase conventions).

```
App\Enum\EventStatus : SCHEDULED , LIVE , COMPLETED , CANCELLED
```

```
App\Enum\MatchStatus : SCHEDULED , CANCELLED , COMPLETED , NO_CONTEST
```

```
App\Enum\CardType : MAIN_CARD , PRELIMS
```

`App\Enum\FightMethod`: `KO`, `TKO`, `UD`, `SD`, `MD`, `PTS`, `DQ`, `TD`, `NC`, `DRAW`

Each class exposes a `getLabel(string $value): string` and `getChoices(): array` method for form usage.

Section 4: Seeded Data

Seeded via SQL within the Phase 1 migration (not via Doctrine fixtures) to guarantee presence in every environment.

Discipline rows (if missing): Boxing, MMA, Judo, Muay Thai.

WeightClass rows (all linked to Boxing discipline, champion NULL):

Order	Name	lbs	kg
1	Heavyweight	—	—
2	Cruiserweight	200	90.72
3	Light Heavyweight	175	79.38
4	Super Middleweight	168	76.20
5	Middleweight	160	72.57
6	Super Welterweight	154	69.85
7	Welterweight	147	66.68
8	Super Lightweight	140	63.50
9	Lightweight	135	61.23
10	Super Featherweight	130	58.97
11	Featherweight	126	57.15
12	Super Bantamweight	122	55.34
13	Bantamweight	118	53.52
14	Super Flyweight	115	52.16
15	Flyweight	112	50.80
16	Light Flyweight	108	48.99
17	Minimumweight	105	47.63

SystemMeta seed: one row with `key = 'last_status_refresh'`, `value = '1970-01-01 00:00:00'`.

No demo fighters, events, or fights are seeded.

Section 5: Migration Strategy

Single atomic migration for all Phase 1 schema changes.

Migration steps in order:

1. Ensure Boxing row exists in `discipline` table
2. Create `weight_class` table
3. Insert 17 weight class rows
4. Create `ranking` table
5. Create `fight_stat` table
6. Create `system_meta` table; insert initial `last_status_refresh` row
7. Alter `fighter`: add `first_name`, `last_name`, `country_code`, `weight_class_id` (new FK), `discipline_id`; change `user_id` to nullable; drop legacy `weight_class_id` (int) and `weight_class` (string); backfill `discipline_id` to Boxing for all existing rows
8. Alter `event`: add `starts_at`, `ends_at`, `venue_name`, `city`, `country`, `poster_url`; backfill `starts_at` from `start_date + 00:00:00`; backfill `ends_at` from `end_date + 00:00:00`; drop `start_date`, `end_date`, `venue_id`, `organizer_id`, `is_champions_event`, `location`; normalize `status` values

- Alter `match_proposal` : add all new fields; backfill `weight_class_id` from fighter's new FK; normalize `status` values ('PENDING' → 'SCHEDULED'); drop `compatibility` if grep confirms unused
- Alter `fight_result` : add `knockdowns_fighter_red`, `knockdowns_fighter_blue`; normalize `method` values to new enum set (map MMA values like 'SUBMISSION' → 'TKO')

Pre-migration requirement: database dump taken and retained. Rollback strategy is restore-from-dump, not Doctrine `down()`.

Schema validation: `doctrine:schema:validate` must pass after migration completes.

Section 6: Registration

Routes

- `GET /register` — display form
- `POST /register` — process submission

Form (RegistrationFormType)

- `email` (EmailType, unique constraint, NotBlank)
- `firstName` (TextType, NotBlank, Length 2-100)
- `lastName` (TextType, NotBlank, Length 2-100)
- `username` (TextType, unique, NotBlank, Length 3-50)
- `password` (RepeatedType, min 8 chars, strength validator)
- `agreeTerms` (CheckboxType, required)

Controller (RegistrationController)

- Hashes password via `UserPasswordHasherInterface`
- Assigns `ROLE_FAN` only — no role selector in UI
- Creates associated `FanProfile` if that pattern is in use (verify during implementation)
- Flash success message, redirect to `/login`

Template

`templates/security/register.html.twig` extends front base, matches login styling.

Security

- CSRF protection (Symfony form default)
- `access_control` entry in `security.yaml` making `/register` publicly accessible
- Rate limiting: deferred unless `symfony/rate-limiter` is already installed

Front UI

"Sign Up" link added next to "Login" in front base template header.

Section 7: CreateAdminCommand

Invocation: `php bin/console app:create-admin`

Interactive prompts

- Email (validated as email, uniqueness checked)
- First name
- Last name
- Username (uniqueness checked)
- Password (hidden input, confirmed twice)

Flags

- `--force` : if email already exists, promote existing user to `ROLE_ADMIN` instead of erroring

Behavior

- Hashes password, creates User with `ROLE_ADMIN`
- Creates associated profile records if pattern applies
- Echoes success message with user ID

Intended use: run once at initial deployment. Subsequent administrators are promoted through an admin-facing UI deferred to a later phase.

Section 8: Status Automation

8.1 RefreshEventStatusCommand

Invocation: `php bin/console app:events:refresh-status`

Per-event logic (skip events with status `CANCELLED`):

```
IF now < event.startsAt:
    target = SCHEDULED
ELSE IF event.startsAt ≤ now ≤ event.startsAt + 6 hours:
    IF all child matches have a linked FightResult:
        target = COMPLETED
    ELSE:
        target = LIVE
ELSE (now > event.startsAt + 6 hours):
    IF all child matches have a linked FightResult:
        target = COMPLETED
    ELSE:
        target = LIVE (stuck — admin needs to enter results)
```

Update event status only if `target ≠ current`. Log status transitions.

After per-event loop, update `system_meta` row `last_status_refresh` to current timestamp.

8.2 StatusRefreshListener

Kernel `onRequest` listener, registered via autoconfigure.

Logic

1. Read `last_status_refresh` from `system_meta`
2. If more than 5 minutes old: run the refresh logic inline (same service as command); update `last_status_refresh`
3. Otherwise, do nothing

Environment control: Listener can be disabled via an env var (`APP_DISABLE_STATUS_REFRESH_LISTENER=1`) when production cron is configured, to avoid double-running.

Performance impact: one indexed SELECT per request when no refresh needed. Negligible.

Section 9: File Inventory

New files

- `src/Entity/WeightClass.php`
- `src/Entity/Ranking.php`
- `src/Entity/FightStat.php`
- `src/Entity/SystemMeta.php`
- `src/Repository/WeightClassRepository.php`
- `src/Repository/RankingRepository.php`
- `src/Repository/FightStatRepository.php`
- `src/Repository/SystemMetaRepository.php`
- `src/Enum/EventStatus.php`
- `src/Enum/MatchStatus.php`
- `src/Enum/CardType.php`
- `src/Enum/FightMethod.php`
- `src/Controller/RegistrationController.php`
- `src/Form/RegistrationFormType.php`
- `src/Command/CreateAdminCommand.php`
- `src/Command/RefreshEventStatusCommand.php`
- `src/Service/EventStatusService.php` (shared logic between command and listener)
- `src/EventListener/StatusRefreshListener.php`
- `templates/security/register.html.twig`

- `migrations/VersionYYYYMMDDHHMMSS_phase1_foundation.php`

Modified files

- `src/Entity/Fighter.php`
- `src/Entity/Event.php`
- `src/Entity/MatchProposal.php`
- `src/Entity/FightResult.php`
- `config/packages/security.yaml`
- `config/packages/vich_uploader.yaml`
- `templates/front/base.html.twig` (or equivalent header partial)

Untouched

- All blog code (controllers, templates, repository, service)
- All prediction code
- All leaderboard code
- All notification code
- All reaction code
- All existing admin screens

Section 10: Delivery Chunks

Phase 1 ships in four reviewable chunks. Each chunk is verified before the next begins.

Chunk 1 — Schema foundation. Migration, new entity classes, modified entity classes, enum classes, repository classes, VichUploader config update.

Chunk 2 — Registration. RegistrationController, RegistrationFormType, register template, security config update, "Sign Up" link in header.

Chunk 3 — Admin command. CreateAdminCommand with interactive prompts and `--force` flag.

Chunk 4 — Status automation. EventStatusService, RefreshEventStatusCommand, StatusRefreshListener, SystemMeta entity already created in Chunk 1.

Section 11: Acceptance Criteria

Phase 1 is complete when all of the following are true:

1. `php bin/console doctrine:migrations:migrate` runs without errors against the existing database
2. `php bin/console doctrine:schema:validate` passes
3. All 17 weight classes are present in the `weight_class` table
4. A visitor can register at `/register` and is assigned `ROLE_FAN`
5. The `app:create-admin` command creates a working administrator account
6. The `app:events:refresh-status` command runs without errors on both empty and populated databases
7. The status refresh kernel listener updates event statuses on request without noticeable latency
8. All existing features continue to work: blog list and detail, prediction submission, leaderboard display, notifications, admin blog CRUD, admin predictions scoring, admin reactions moderation, admin notification broadcast
9. No existing routes produce 500 errors after migration

Section 12: Known Risks & Mitigations

Risk 1: Event date migration loses time information

Detail: Existing `startDate` is date-only; conversion defaults times to `00:00:00`.

Mitigation: Accept for existing test data; administrators re-enter times where needed. Production hasn't launched, so the population of affected events is small.

Risk 2: Method value migration for MMA → boxing creates semantic mismatches

Detail: Existing `FightResult` rows may contain 'SUBMISSION' or other MMA methods invalid under the new enum.

Mitigation: Migration maps ambiguous values to closest boxing equivalents (e.g., 'SUBMISSION' → 'TKO') and logs the mapping. Administrators can correct edge cases post-migration.

Risk 3: Backfilling MatchProposal.weightClass from fighter data

Detail: Existing match proposals have no weight class and the FK is NOT NULL.

Mitigation: Migration attempts to derive from the fighters involved; if fighters disagree on weight class, the field is left NULL and the NOT NULL constraint is added only after a manual cleanup pass. Alternative fallback: mark the column NULL-able in this migration and tighten it in Phase 2.

Risk 4: Kernel listener fires on every request

Detail: Even with the 5-minute throttle, the SELECT on `system_meta` runs on each request.

Mitigation: Column is the primary key or uniquely indexed; query cost is negligible. Can be disabled via env var if true cron is set up.

Risk 5: compatibility field may still be in use somewhere

Detail: Drop assumes grep during implementation confirms unused status.

Mitigation: If usage is found, the field is retained and the Phase 1 migration omits the DROP.

Section 13: Deferred to Later Phases

Explicitly out of scope for Phase 1:

- Admin CRUD for fighters, events, matches, results (Phase 2)
- Public events listing and detail pages (Phase 3)
- Fight stats admin UI with grid entry and CSV import/export (Phase 4)
- Rankings admin UI and public rankings page (Phase 5)
- Administrator-facing UI for promoting users to administrator
- Email verification on registration
- Password reset flow
- Fighter search/autocomplete
- Internationalization (currently English-only)

Sign-off. All decisions in this document reflect agreed recommendations. When you're ready, implementation begins with Chunk 1: schema foundation.